

# Building Without Bytes

## High-Velocity Image Composition at Scale

These slides were not exported from an app — they were **built by Bazel**, from the same workspace that builds the containers they describe.

Sponsored by **Tweag by Modus Create**

BazelCon 2026 · 9:00–10:30 · Rafael Lima

# This slide is a build artifact

`bazel build //slides:build` → this deck.
`bazel build //samples/python:image` → the image we'll discuss.

Same graph. Same cache. Same guarantees.

What you build is byte-for-byte what runs in production —
starting with the deck you're watching.

# Module 0 · Why: correct & hermetic

10 min · Lab: clone + `bazel build //...`

# Bazel’s promise:

**Correct** means **hermetic** — a build with *no external influence*, so identical inputs always produce identical outputs.

Reproducibility isn’t bolted on; it’s the whole point. Everything in this workshop is a consequence of taking that seriously for **container images**.

# Two kinds of hermeticity

## Docker

**Runtime / deploy** hermeticity.
Bundles deps so the artifact runs the same everywhere.
But `docker build` itself is **not** hermetic — `RUN apt-get` is networked, timestamped, order-sensitive.

## Bazel

**Build** hermeticity.
Only declared inputs are visible; identical inputs → identical outputs.
Emits a reproducible, content-addressed artifact.

Docker hermetizes the **deploy** boundary; Bazel the **build**. Want both.

# The "black box" build

Standard Dockerfiles are imperative scripts — and that hurts three ways:

- **Non-deterministic** — `apt-get` today ≠ `apt-get` tomorrow.
- **Coarse cache invalidation** — one early edit busts every layer below.
- **The Docker socket** — a privileged daemon: security surface + CI bottleneck.

Bazel's answer: stop **running** commands to make layers — **assemble** them from pre-built, content-addressed artifacts. The filesystem becomes data.

# Module 1 · A first declarative image

15 min · Lab: build/run the image vs the Dockerfile

# The "before": an imperative Dockerfile

The Docker daemon installs the `dot` system dependency with `RUN apt-get` — an opaque, stateful layer. Editing `app.py` busts everything below the `COPY`.

```
# The "before": a traditional, imperative Dockerfile build (TASK §2.2).
#
# Note the system dependency handled the daemon way: `apt-get install graphviz`
# in a RUN layer. It works, but the layer is an opaque blob produced by a
# stateful daemon, and editing app.py below busts every layer after the COPY.
# Compare with //samples/python:image, where the same dependency is a
# declarative, content-addressed layer in the Bazel action graph.
#
# Build & run:
#   docker build -t bwb/python-graphviz-dockerfile samples/python
#   docker run --rm bwb/python-graphviz-dockerfile   # renders the PNG

FROM python:3.12-slim-bookworm

# System dependency: the `dot` binary. Imperative, daemon-run, cache-coarse.
RUN apt-get update \
    && apt-get install -y --no-install-recommends graphviz \
    && rm -rf /var/lib/apt/lists/*

# Python dependency layer.
COPY requirements_lock.txt /tmp/requirements.txt
```

# The payload: Python + a *non-hermetic* system dep

`graphviz` shells out to the `dot` binary — the apt dependency we must make explicit once we leave the daemon. The app renders the workshop's own graph:

```python
def build_graph() -> graphviz.Digraph:
    """The supply chain that builds this very talk (the dogfooding loop)."""
    dot = graphviz.Digraph("supply_chain", comment="Building Without Bytes")
    dot.attr(rankdir="LR")
    dot.attr("node", shape="box", style="rounded")

    dot.node("src", "source\n(app.py, BUILD)")
    dot.node("deps", "pip deps\n(graphviz)")
    dot.node("sysdep", "system layer\n(dot binary)", style="rounded,filled", fillcolor="lightyellow")
    dot.node("image", "oci_image")
    dot.node("slides", "slides deck")
    dot.node("registry", "registry")

    dot.edge("src", "image")
    dot.edge("deps", "image")
    dot.edge("sysdep", "image")
    dot.edge("image", "registry")
    dot.edge("src", "slides", label="transclude")  # the deck imports this file
    return dot
```

# The "after": layers as declarative build outputs

No daemon. The image is composed from content-addressed layers in the action graph; changing `app.py` reuses the base + deps layers untouched.

```
oci_image(
    name = "image",
    base = "@python_base",
    entrypoint = [
        "/app/app",  # py_binary bootstrap launcher; locates /app/app.runfiles
        "--render",
        "/tmp/supply_chain.png",
    ],
    tars = [":app_layer"],
)
```

# Module 2 · Hermeticity in practice

15 min · Lab: the "docker tax" benchmark

# Is Docker a good hermeticity validator?

As a **negative test** — yes:

```
docker run --network=none --rm myscript   # fails if it needs the host/network
```

A pinned base with no network *surfaces undeclared dependencies*. But it's **not a proof** — build-time network, timestamps, and ordering still sneak in.

Bazel's **sandbox** is the stronger validator: it exposes only *declared* inputs, so a missing dependency fails the build — every time.

# The "docker tax"

Same suite, two ways — `py_test` (sandboxed, cached, incremental) vs. a `docker build` that pip-installs and runs pytest. Change one line, re-run:

```
bazel run //benchmark/pytest_tax:compare
#   bazel test  (warm, after edit)    ~0.8 s
#   docker build (warm, after edit)   ~2.2 s   ← per-edit "tax"
#   docker build (cold, --no-cache)  ~11   s   ← upfront "make it hermetic" cost
```

Absolute numbers are machine-dependent; the **ratio per edit** is the point — and it grows with image size and dependency count.

# Module 3 · Layer optimization

25 min · Lab: layer split · dedup · distroless

# Architect layers by volatility

Order layers least → most volatile so rebuilds stay cheap:

1. Base OS — static · 2. Runtimes / deps — infrequent · 3. App code — constant

Change one string in a multi-GB app:

```
$ docker build -t app .              # invalidates at COPY . /app → 2–3 min
$ bazel run //samples/rules_img:load # re-assembles one blob   → seconds
```

Bazel re-assembles only the changed layer — and uploads only that blob.

# Action deduplication

Bazel's action cache keys every action on **input digests + command + env** → identical action, built **once**.

Layers are **content-addressed**, so the win compounds:

- a shared base/deps layer is **built once**, reused across *every* image;
- at the registry, an existing blob is **skipped on push**.

Optimization = make common layers **byte-identical**, and dedup does the rest.

# rules_distroless: hermetic system deps

Remember the `dot` gap? Replace `RUN apt-get install graphviz` with a **pinned manifest + lockfile** — Bazel materializes `dot` + its `.deb`s as content-addressed layers. No network at build, no daemon, no `RUN`.

```
# MODULE.bazel
apt = use_extension("@rules_distroless//apt:extensions.bzl", "apt")
apt.install(name = "graphviz_apt",
            manifest = "//samples/distroless:graphviz.yaml",
            lock = "//samples/distroless:graphviz.lock.json")
```

Bonus: **no shell, no package manager** → smaller image, smaller attack surface.

# Module 4 · Push operations

15 min · Lab: push to a local registry

# Daemonless push — only the changed blobs

`rules_oci` → `oci_push`; `rules_img` → `image_push`. No daemon. Both upload **only blobs the registry lacks** — the dedup from Module 3 now pays off as **bandwidth**: a shared base layer uploads once.

```
bazel run //tools:local_registry          # registry:2 on localhost:5000
bazel run //samples/rules_oci:push         # push (changed blobs only)
bazel run //samples/rules_img:push         # same image, rules_img
```

Push a multi-arch **index** (`linux/amd64` + `linux/arm64/v8`) — no QEMU-in-Docker.

# Module 5 · rules_img & shipping choices

10 min · wrap-up + Q&A

# Same artifact, three philosophies

| Feature | Dockerfile | rules_oci | rules_img |
|---|---|---|---|
| **Execution** | Imperative | Declarative | Optimized declarative |
| **Hermetic** | No | Yes | Yes |
| **Speed** | Slow (cache miss) | Fast | Ultra-fast |
| **Daemon req.** | Yes | No | No |
| **Tar-less** | No | No | Yes |

`rules_img` skips `tar`/compression and uses a Go-based incremental loader — layers stay transparent blobs until final assembly.

# Ship as Docker, or as source?

## Docker image

Self-contained, immutable, content-addressed, scannable, rollback-able.
Carries **deploy hermeticity**.

## Source only

Smaller to transfer — but pushes non-hermeticity onto the consumer's build ("works on my machine" returns).

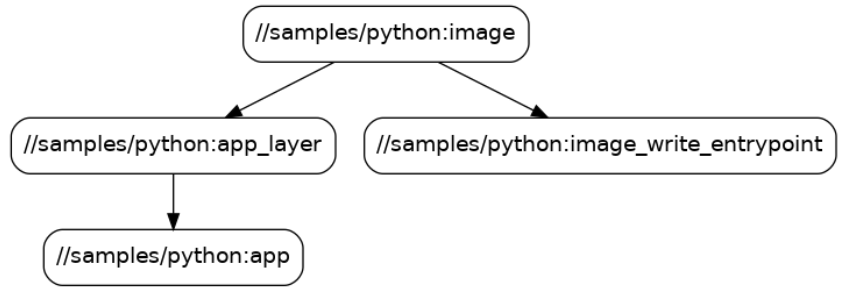
With Bazel you emit **both from one graph** — choose per audience.

# The dogfooding loop

The Python sample renders **this repo's own dependency graph** with `dot`, and the deck imports the result.

A Bazel action, needing an apt-installed binary packaged as a layer, draws the graph of the build that produced the slide.

```
bazel run //tools:render_graph \
  -- //samples/python:image
```



# Takeaways

**Stop scripting, start mapping** — declare *what goes where*, not what to run.
**Architect for layers** — heavy deps apart from volatile code.
**Go daemonless** — better CI security and portability.
**Evaluate** `rules_img` — if `tar` is the bottleneck, upgrade.
**Slide-as-Code** — the deck is just another build target.

Once layers are just artifacts, the daemon stops being special.